

Indexing Transformations for Relational Symbolic Trajectory Evaluation

1075201

A dissertation submitted for the degree of
Computer Science (Part B)
Trinity 2025

Word Count: ?

1. Abstract

Symbolic Trajectory Evaluation (STE) is a symbolic model checking method for hardware verification that is able to exploit abstraction via symbolic indexing to perform verification efficiently. Relational STE is a modification of the STE method to allow for arbitrary relational properties to be verified. This project reinvents the theory of symbolic indexing transformations to allow rSTE to exploit symbolic indexing abstractions in a manner similar to regular STE. We further update existing theory for efficient indexing transformations on a subclass of abstractions called partitioned abstractions to deal with symbolic constants and discuss methods for dealing with environmental constraints. These procedures are implemented in the industry standard verification tool JasperGold and our experiments show them to be efficient on scalable example circuits compared to symbolic simulation without symbolic indexing.

Contents

1. Abstract	1
2. Introduction	4
2.1. Background	4
2.2. Contributions	7
2.3. Relation to Companion Project	7
3. Theory for Relational STE	7
3.1. Design Under Test, Specifications and Relational Properties	7
3.1.1. Relational Properties with SVA	7
3.1.2. Output Constraints	8
3.1.3. Antecedents	9
3.2. Indexing Relations and Transformations	9
3.2.1. Preimage Operations	10
3.2.2. Image Operation	10
3.2.3. Relationship between Preimages	10
3.2.4. Indexing Coverage Condition	11
3.2.5. Constructing Indexing Relations	11
3.3. rSTE Model Checking Procedure	11
3.4. Simplified Checking for Unrestricted Output Constraints	12
3.5. Soundness of Model Checking Procedure	13
3.5.1. Symbolic Simulation Invariants	13
3.5.2. Output Constraint Checking	15
3.5.3. Alternative Checking by Reversing the Indexing	16
3.5.4. Counter Example Analysis	17
4. Partitioned Abstraction Relations	18
4.1. Efficient Weak Preimage Computation	18
4.2. Efficient Strong Preimage Computation	21
5. Verification Under Environmental Constraints	21
5.1. Index Over Care Predicate Cases via Indexing Relation Restriction	21
5.1.1. Preserve Partitioned Indexing Relation by Conditioning Antecedent	22
5.2. Parametric Encoding	22
5.2.1. Parametric Encoding of Indexing Relations	23
5.2.1.1. Equivalence to Indexing Relation Restriction	25
5.2.2. Parameterise Before Abstraction	26
5.3. Conditioned Properties	26
6. Experiments and Evaluation	27

6.1. Content-Addressable Memory (CAM)	27
6.1.1. Manual Indexing Relation	28
6.1.2. Results	29
6.2. Multi-Input Maximum Circuit	29
6.2.1. Manual Indexing Relation	30
6.2.2. Results	32
7. Conclusion	32
7.1. Future Work	32
Bibliography	33

2. Introduction

2.1. Background

Formal verification is the process of proving that a system satisfies a given specification or set of properties. Symbolic Trajectory Evaluation (STE) [1], [2] is a method to perform hardware formal verification on circuits that allows one to verify that circuits satisfied functional properties while incorporating abstraction via symbolic indexing.

The core idea of STE is doing circuit simulation over a 3-valued domain of “true”, “false” and “don’t know” (X) that is ordered based on information content. On any concrete set of inputs over the 3-valued domain, a simulator would propagate these values along the circuit, processing each logical gate based on a modification of its truth table to account for the “don’t know” value. For example, an logical AND gate would have a truth table as in Table 1

AND	0	1	X
0	0	0	0
1	0	1	X
X	0	X	X

Table 1: 3-valued excitation function for AND gate

We can use this 3-valued simulation to observe behaviours of circuits on groups of related inputs. For example, by simulating the above AND gate on the inputs 0 and X, we can observe that the output is 0 and thus conclude that $0 \wedge 0 = 1 \wedge 0 = 0$. This has allowed us to merge the analysis of 2 cases that we would have to simulate separately in a traditional 2-valued boolean simulation.

Furthermore, rather than doing simulation of only 1 concrete instance of 3-valued inputs at once, we can make the simulator work symbolically to do multiple 3-valued simulations at once. In the case of basic boolean simulation, we could start by assigning a variable to each input and then propagate these values through the circuit, building up a propositional formula at each node that represents the truth value of the circuit node.

For our 3-valued simulation, we do this symbolic simulation using a pair of boolean formulae over “indexing variables” on each circuit node. For each node, these are called

the “high” and “low” expressions and have the following semantics.¹

	$H_s = 0$	$H_s = 1$
$L_s = 0$	s is X	s is 1
$L_s = 1$	s is 0	s is $\top$

Table 2: Semantics of High and Low Expressions

Where $\top$ represents an inconsistency.

The symbolic simulator propagates these high and low expressions through the circuit. For example, for an AND gate, on inputs with expressions (h_A, l_A) and (h_B, l_B) , the output produced would be $(h_A \wedge h_B, l_A \vee l_B)$.

Symbolic simulators generally represent these expressions as reduced, ordered binary decision diagrams (BDDs) [3] that allow for efficient manipulation and thus simulation of the circuit. We term these pairs of BDDs as dual rail BDDs.

As an example, consider verification of a 3-input AND gate. We would like to prove that $o = a \wedge b \wedge c$ for the inputs a, b, c . To use STE to verify the AND gate, we observe that rather than simulating all 2^3 different boolean input assignments to the circuit, it is sufficient to simulate 4 cases under the 3-valued domain:

$$\begin{aligned}
p \wedge q : a = b = c = 1 \\
\bar{p} \wedge \bar{q} : a = 0, b = c = X \\
\bar{p} \wedge q : b = 0, a = c = X \\
p \wedge \bar{q} : c = 0, a = b = X
\end{aligned}$$

We can enumerate these cases symbolically by assigning them propositional formulae in terms of indexing variables p and q . Using propositional formulae to do case splitting this way is termed as symbolic indexing.

To apply this symbolic indexing scheme, we will set the dual rail inputs as follows:

¹This implementation strategy is the modern one implemented in JasperGold. The legacy literature[2] had different names and interpretations for the two expressions.

$$h_a := p \wedge q, l_a := \bar{p} \wedge \bar{q}$$

$$h_b := p \wedge q, l_b := \bar{p} \wedge q$$

$$h_c := p \wedge q, l_c := p \wedge \bar{q}$$

We then run the symbolic simulator, getting output $(h_o, l_o) = (p \wedge q, \bar{p} \vee \bar{q})$. We can then conclude that the AND gate outputs true if and only if $a = b = c = 1$ and false otherwise, which is the desired property.

Legacy literature on STE [1] provided theory on writing stimuli (the antecedent) to combinational circuits using a linear temporal logic, specifying an intended functional output (consequence) and then using the symbolic simulator to verify that the circuit would satisfy the output under the given stimuli. However, it was difficult to write stimuli in a manner that covered all relevant input cases and to interpret the results of the symbolic simulation to check properties. As such, the theory of indexing relation transformations [4] was later developed to simplify usage of STE.

To use STE with indexing transformations, we would first set up an antecedent and consequent without any abstraction from Xs, i.e. the for each input $h_i = \neg l_i$. We then apply a transformation on the antecedent and consequence based on an indexing relation that mapped groups of target assignments to corresponding indexing variable assignments. This would provide us with symbolic indexing in a systematic way that covered all relevant input cases, assuming a technical coverage condition on the indexing relation was met. We could then check that the transformed antecedent would lead to the transformed consequence holding on the circuit.

Further work was later done to automate the process of generating this indexing relation via an automatic abstraction algorithm [5].

However, STE never quite caught on and the tools that supported it such as Intel's Forte [6] and VOSS [7] were eventually deprecated. This was mainly due to the limitation in writing functional specifications and difficulties in applying the technique to cases with environmental constraints on the inputs.

Since then, a variant of STE known as relational STE (rSTE) [8] that allows for specifications with arbitrary relational properties has been described but not formalized. Furthermore, a dual-rail BDD symbolic simulator has been developed within the industry standard formal verification tool JasperGold [9].

This project thus reinvents theory for rSTE and brings it to the modern day in JasperGold.

2.2. Contributions

We reinvent the theory of rSTE, providing a procedure for incorporating symbolic indexing into the rSTE workflow via indexing transformations. We provide proofs of soundness for the procedure of indexing transformations and interpretations of the simulation outputs to derive proofs and counterexamples.

We provide new proofs and formal descriptions for efficient preimage computations on a subclass of indexing relations known as partitioned indexing relations that extends the existing technique by incorporates symbolic constants mentioned in [5].

We formulate theory for dealing with environmental constraints in rSTE using ideas from [4] but updated to consider partitioned indexing relations with symbolic constants.

We implement the theoretical procedures to perform rSTE with symbolic indexing in JasperGold, including procedures for efficient indexing transformations of partitioned indexing relations, calling of symbolic simulator with transformed antecedent, and checking of transformed output.

We evaluate the efficiency of rSTE on two scalable example circuits: a Content-Addressable Memory (CAM) and a multi-input maximum circuit. For each of these, we evaluate the technique with the use of a manually constructed indexing relation and an automatically generated one when compared to symbolic simulation without symbolic indexing. We used a manual indexing of the CAM based on [10] and crafted an original one for the maximum circuit.

2.3. Relation to Companion Project

While this project focuses on the process of using indexing relations to do rSTE with symbolic indexing, the companion project focuses on the automatic generation of indexing relations for rSTE. The two projects are complementary and can be used together to perform rSTE with minimal user input.

3. Theory for Relational STE

3.1. Design Under Test, Specifications and Relational Properties

We make use of JasperGold to process a circuit design under test (DUT) written in SystemVerilog at Register Transfer Level (RTL).

3.1.1. Relational Properties with SVA

Traditionally STE properties were functional properties that were written in a linear temporal logic that allowed you to assert that specific signals would be high or low depending on the specific stimulus on the input of the circuit.

However, rSTE allows for more general properties through the use of a binded specification circuit that implements properties written as SystemVerilog Assertions (SVA). The specification circuit would be binded in the sense that when the DUT was elaborated, the specification circuit would be instantiated together with it and take as input, both the inputs to the DUT, as well as any internal/output signals from the DUT that the properties desired would depend on.

The key to checking relational properties is then the insight that SVA properties can be expressed as a circuit that produces a signal that is high if the property holds in that clock cycle and low otherwise. This is done as part of JasperGold’s internal symbolic simulation model construction. We call the signal that represents the property being held as the “property wire”.

When using the symbolic simulator without symbolic indexing, all we need to do is to do a symbolic simulation and check that the property wire is high under any assignment of inputs.

3.1.2. Output Constraints

To incorporate symbolic indexing via indexing transformations, we build an additional layer of output constraints on top of the SVA properties that we want to check. These output constraints will mirror the LTL properties from STE but are written in a 5-tuple form as implemented in VOSSII [11]:

Output Constraint: [(signal, tickStart, tickEnd, highExpr, lowExpr)]

This output constraint means that on cases where `highExpr` is true, `signal` should be high. When `lowExpr` is true, `signal` should be low. If neither are true, then we don’t assert anything about the signal. The `tickStart` and `tickEnd` are the clock cycles that the output constraint should hold for. These expressions are known as the guards of the output constraint.

Observe that each output constraint thus consists of two parts: the positive output constraint, which enforces that signal must be high under some condtions, and the negative output constraint, which enforces that signal must be low under some conditions.

Before doing the symbolic indexing, for each SVA property we wish to check, we will build a corresponding output constraint of the form:

Output Constraint: [(propertyWire, tickStart, tickEnd, True, False)]

Which indicates that the SVA property should always hold. Such output constraints are termed as “unrestricted output constraints” since they are always expected to hold and never expected to be false.

3.1.3. Antecedents

The 5-tuple form is also the form that the antecedent (stimulus) of the circuit will be written in. When a tuple (`signal`, `tickStart`, `tickEnd`, `highExpr`, `lowExpr`) is written in the stimulus, it means that the signal should be high when `highExpr` is true, low when `lowExpr` is true and X or inconsistent otherwise.

When we do symbolic simulation without symbolic indexing, for each input signal s and clock cycle t that we wish to provide a unique stimulus to s for, we will include a tuple (s , t , t , $z_ (s, t)$, $\text{NOT } z_ (s, t)$) in the antecedent. This tuple ensures a new unique BDD variable $z_ (s, t)$ is created, ensuring that we cover all possible input cases. We would then run the symbolic simulator with this antecedent and check that high expression of the property wire is true for all relevant clock cycles.

3.2. Indexing Relations and Transformations

Indexing relations are used to symbolic represent input case splitting to perform symbolic indexing and are the basis of the indexing transformation procedure.

Indexing relations $R[X, C, T]$ are boolean formulae over BDD variables where

- X are the indexing BDD variables that perform the symbolic indexing.
- T are the target BDD variables that are indexed. These correspond to input wires in the circuit.
- C are the symbolic constants that correspond to input wires that should not be indexed.

The indexing relation $R[X, C, T]$ can be interpreted as follows:

- For each X, C , we cover the cases T, C where $\exists T R[X, C, T]$.
- Since the indexing relation can be a many to many relation, we can have multiple T, C cases covered by a single X, C case indexing and vice versa.

Note that the indexing relation $R'[(X, C'), \emptyset, (C, T)] = R[X, C, T] \wedge \bigwedge (C = C')$ that doesn't use symbolic constants is equivalent to $R[X, C, T]$ from the perspective of doing symbolic simulation and its associated operations (Section 3.2.1). The use of symbolic constants does not improve the expressive power of indexing relations, but rather simplifies them to achieve higher efficiency. When using the automatic abstraction algorithm in the companion paper, symbolic constants provide a crude way to prevent over abstraction since the algorithm will not abstract over them.

3.2.1. Preimage Operations

We define operations involving the indexing relation here:

Suppose

- $P[C, T]$ represents some set of input cases (e.g. (C, T) is in the set iff $P[C, T]$ is true)
- $R[X, C, T]$ is an indexing relation

The **weak preimage** of P under R ,

$$P_R[X, C] = \exists T(R[X, C, T] \wedge P[C, T])$$

is the set of (X, C) that cover at least one (T, C) case in P

The **strong preimage** of P under R ,

$$P^R[X, C] = P_R \wedge \neg \exists T(R[X, C, T] \wedge \neg P[C, T])$$

is the set of (X, C) that cover at least one case in P and don't cover any cases not in P .

3.2.2. Image Operation

Taking the image of $H[X, C]$ returns the cases that are indexed by $H[X, C]$ under R .

$$\text{im}(H, R)[C, T] = \exists X(R[X, C, T] \wedge H[X, C])$$

While this is not directly used for finding proofs, it is useful for extracting counter examples for disproven properties.

3.2.3. Relationship between Preimages

For any fixed R and any given P and (X, C) we have one of the following cases:

- X, C indexes cases only in P
- X, C indexes cases only in $\neg P$
- X, C indexes cases in both P and $\neg P$
- X, C indexes cases in neither P nor $\neg P$

Any X, C that fall into the 4th category must correspond to empty indexing cases, i.e. don't map onto any (C, T) at all. Notice that this is independent of P , thus, we term the union of the first 3 cases as the domain of R .

$$\text{dom}(R)[X, C] = \exists T(R[X, C, T])$$

Observe that

- The weak preimage is the set of X, C that fall into the 1st and 3rd categories.

- The strong preimage is the set of X, C that fall into the 1st category.

We can note that the following are then also equivalent definitions of the strong preimage operation:

$$\begin{aligned} P^R[X, C] &= \text{dom}(R) \wedge \overline{P_R} \\ &= \text{dom}(R) \wedge (\forall T (R[X, C, T] \rightarrow P[C, T])) \end{aligned}$$

3.2.4. Indexing Coverage Condition

For each output constraint (Section 3.1.2) i that we wish to check, we let the high and low expressions be P_i and Q_i respectively. The total space of inputs we need to check is then $B[C, T] = \bigvee_i (P_i \vee Q_i)$. In order for our abstraction to properly cover the space of required inputs, we require that

$$\forall T \forall C (B[C, T] \rightarrow \exists X (R[X, C, T]))$$

For unrestricted output constraints, we will have $P_i = \text{True}, Q_i = \text{False}$. In this case, $B[C, T] \equiv \text{True}$ so the coverage condition takes the simpler form:

$$\forall T \forall C \exists X (R[X, C, T])$$

Intuitively, the coverage condition states that each input assignment of target variables must be covered by some indexing case and thus considered by the model checker.

3.2.5. Constructing Indexing Relations

To formulate the indexing relation, we will need to encode the case splitting for the different input cases that elicit different outputs from the circuit. This is done manually for specific circuits in the examples of Section 6.1 and Section 6.2.

The companion project reinvents the automatic abstraction algorithm from [5] and implements it within JasperGold. It produces a partitioned indexing relation (Section 4) that satisfies the coverage condition by construction.

3.3. rSTE Model Checking Procedure

Suppose we are given an indexing relation R , an antecedent list `antv` and an output constraint list `cout`. We will describe the procedure to prove that the output constraints hold.

We first apply the strong preimage operations on the high and low expressions of the antecedent tuples as follows:

$$\text{antv} = [(\text{signal}, \text{tickStart}, \text{tickEnd}, \text{highExpr}^R, \text{lowExpr}^R)]$$

Next we run the symbolic simulator with the transformed antecedent. This produces an evaluation sequence which will contain `highExpr` and `lowExpr` BDD expressions for when a signal at a certain time step will definitely be high or low. If for a given assignment, neither of these expressions are true, then the signal is unknown. For a given signal and time step, we let the high expression be H and L .

From this evaluation sequence, we can then check the output constraints. Suppose want to check a output constraint of the form `(signal, tickStart, tickEnd, P, Q)` where $P[C, T]$ and $Q[C, T]$ are the BDD expressions that correspond to inputs on which a signal should be high and low respectively. We will transform the output constraint to

$$(\text{signal}, \text{tickStart}, \text{tickEnd}, P_R, Q_R)$$

where P_R and Q_R are the weak preimages of P and Q under the indexing relation.

Then we analyse the `highExpr` and `lowExpr` BDD expressions from the evaluation sequence for the signal at each of the ticks in the tick range. Suppose that at a given tick t , for the property wire we have some the high expression as $H[X, C]$ and the low expression as $L[X, C]$. We term the high expression of the signal as the “residual” [2].

We will check if $(P_R \rightarrow H)[X, C] \equiv \text{True}$ and whether $(Q_R \rightarrow L)[X, C] \equiv \text{True}$. If both of these hold, then the output constraint is satisfied.

If we have $(P^R \wedge L) \neq \text{False}$ it means that some indexing cases that only index into P actually cause s to be low, this is a counter example to the positive part of property so the positive part of property is disproven. We can inspect the cases indexed as $\text{im}(P^R \wedge L, R)$ to get the actual counter example in terms of the original circuit inputs.

Similarly, if we have $(Q^R \wedge H) \neq \text{False}$, we have disproven the negative part of the property.

It is possible to not prove the property but also have no counter examples. This can happen in cases where the antecedent does not provide enough information to prove or disprove the property. This is called a weak disagreement [2] and requires refining the indexing relation to obtain a proof/counter example.

3.4. Simplified Checking for Unrestricted Output Constraints

A benefit of the unrestricted output constraint form is that the property guards are extremely simple and thus have some nice properties related to the indexing transformation.

To check the properties, we need to know the weak/strong preimage images of the guards. However, we have the following:

$$\text{True}_R = \text{True}^R = \text{dom}(R)[X, C]$$

$$\text{False}_R = \text{False}^R = \text{False}$$

This means that we only need to take 1 single preimage operation to get the domain and use that for checking all the various properties written in this form. In fact, computing the domain of a partitioned indexing relation (Section 4) is a very simple operation that will be described later.

Furthermore, analysis of counter examples is also simplified.

$$\text{True}^R \wedge L = \text{dom}(R)[X, C] \wedge L = L$$

The 2nd equality comes from the fact that we use strong preimages of target variables and their negations to transform the antecedents, so for any signal and time step, H and L do not include any cases that are not in the domain of R . I.e. $H \vee L \rightarrow \text{dom}(R)$. This can be proven with induction over the circuit in a similar manner as the symbolic simulation invariants in Section 3.5.

So any time L is not false, we have found a counter example. Specifically, any T, C in $\text{im}(L, R)$ is a counter example.

3.5. Soundness of Model Checking Procedure

We prove that the model checking procedure described in Section 3.3 is sound.

3.5.1. Symbolic Simulation Invariants

Let $\text{st}(C, T)$ be true iff the signal and time pair st is high under the untransformed antecedent under some assignment of the variables in C, T .

Consider the low and high expressions in any signal timestep pair. We always have that

$$H_{\text{st}}[X, C] \wedge R[X, C, T] \rightarrow \text{st}(C, T)$$

$$L_{\text{st}}[X, C] \wedge R[X, C, T] \rightarrow \overline{\text{st}(C, T)}$$

(For all X, C, T)

This can be proved via induction on the fan-in of st .

The main basecase is where s is an input variable corresponding to a certain time step, with `highexpr` P and `lowexpr` $\overline{P}$. In this case, since we apply the strong preimage to the antecedents, we have that

$$\begin{aligned} H &= P^R = \text{dom}(R) \wedge \forall T (R[X, C, T] \rightarrow P[C, T]) \\ L &= \overline{P}^R = \text{dom}(R) \wedge \forall T (R[X, C, T] \rightarrow \overline{P[C, T]}) \end{aligned}$$

This directly gives us that for all X, C, T where $H[X, C] \wedge R[X, C, T]$, we have $P[C, T]$. But $\text{st}(C, T) = P[C, T]$ since P will be a target variable / symbolic constant corresponding to the stimulus to the signal at the time step in the untransformed antecedent (Section 3.1.3), so we are done.² The proof for $L[X, C] \wedge R[X, C, T] \rightarrow \overline{P[C, T]}$ is analagous.

A more subtle basecase is when we consider a circuit node with state, such as a latch, before any stimulus from the antecedent reaches it. In this case, the high and low expressions will be false so the invariants hold trivially.

An inductive case example:

Suppose $s = s_1 \wedge s_2$. Since the AND gate is combinational, we fix any time step. Let (H_{s_1}, L_{s_1}) and (H_{s_2}, L_{s_2}) be the high and low expressions for s_1 and s_2 respectively. The symbolic simulator will give us $(H_s, L_s) = (H_{s_1} \wedge H_{s_2}, L_{s_1} \vee L_{s_2})$. Under any fixed X, C, T , Figure 1 gives us a proof that $H_s[X, C] \wedge R[X, C, T] \rightarrow s(C, T)$ and $L_s[X, C] \wedge R[X, C, T] \rightarrow \neg s(C, T)$.

Other elementary circuit components can be proven in a similar manner.

²Note that we don't use the domain part for this. Indeed, the analysis will still work, but this serves to remove useless/inconsistent indexing cases. This is useful for counterexample analysis later on.

	1.	$H_s[X, C]$	Assume
	2.	$R[X, C, T]$	Assume
1	3.	$H_{s_1}[X, C] \wedge H_{s_2}[X, C]$	$H_s = H_{s_1} \wedge H_{s_2}$
	4.	$H_{s_1}[X, C] \wedge R[X, C, T] \rightarrow s_1(C, T)$	Inductive Hypothesis
	5.	$H_{s_2}[X, C] \wedge R[X, C, T] \rightarrow s_2(C, T)$	Inductive Hypothesis
2, 3, 4	6.	s_1	
2, 3, 5	7.	s_2	
6, 7	8.	s	$s = s_1 \wedge s_2$
1, 2, 8	9.	$H_s[X, C] \wedge R[X, C, T] \rightarrow s(C, T)$	Conclusion

	1.	$L_s[X, C]$	Assume
	2.	$R[X, C, T]$	Assume
1	3.	$L_{s_1}[X, C] \vee L_{s_2}[X, C]$	$L_s = L_{s_1} \vee L_{s_2}$
	4.	$L_{s_1}[X, C] \wedge R[X, C, T] \rightarrow \neg s_1(C, T)$	Inductive Hypothesis
	5.	$L_{s_1}[X, C]$	Assume
2, 6, 4	6.	$\neg s_1(C, T)$	
7	7.	$\neg s$	$s = s_1 \wedge s_2$
	8.	$L_{s_2}[X, C] \wedge R[X, C, T] \rightarrow \neg s_2(C, T)$	Inductive Hypothesis
	9.	$L_{s_2}[X, C]$	Assume
3, 6, 5	10.	$\neg s_2(C, T)$	
7	11.	$\neg s$	
3, 5-7, 9-11	12.	$\neg s$	
1, 2, 8	13.	$L_s[X, C] \wedge R[X, C, T] \rightarrow \neg s(C, T)$	Conclusion

Figure 1: Inductive Case on AND Gate

3.5.2. Output Constraint Checking

Positive output constraints are of the form:

$$\forall C \forall T (P[T, C] \rightarrow s(C, T))$$

If we have that $P_R[X, C] \rightarrow H[X, C] \equiv \text{True}$ and the coverage condition $\forall T \forall C (B[C, T] \rightarrow \exists X (R[X, C, T]))$ holds, then we know the output constraint will hold as shown in Figure 2.

	1.	Fresh X, C, T	
	2.	$P[C, T]$	Assume
	3.	$R[X, C, T]$	Assume
2, 3	4.	$P[C, T] \wedge R[X, C, T]$	
4	5.	$\exists T(P[C, T] \wedge R[X, C, T])$	
5	6.	$P_R[X, C]$	Definition of P_R
			Since X, C, T were
1, 6	7.	$\forall X, C, T (P[C, T] \wedge R[X, C, T] \rightarrow P_R[X, C])$	arbitrary, lemma

	1.	Fresh C, T	
	2.	$\forall X(P_R[X, C] \rightarrow H[X, C])$	Premise
	3.	$B[C, T] \rightarrow \exists X(R[X, C, T])$	Premise
	4.	$P[C, T]$	Assume
	5.	$\forall X, C, T (P[C, T] \wedge R[X, C, T] \rightarrow P_R[X, C])$	Lemma above
4	6.	$B[C, T]$	Definition of $B[C, T]$
3, 6	7.	Fresh X^* s.t. $R[X^*, C, T]$	
4, 5, 7	8.	$P_R[X^*, C]$	
2, 8	9.	$H[X^*, C]$	
			Symbolic Simulation
9	10.	$s(C, T)$	Invariants
4, 10	11.	$P[C, T] \rightarrow s(C, T)$	
			Since C, T were
1, 11	12.	$\forall C \forall T (P[C, T] \rightarrow s(C, T))$	arbitrary

Figure 2: Proof of Positive Output Constraint Check

We can prove the check for negative output constraints of the form

$$\forall C \forall T (P[C, T] \rightarrow \neg s(C, T))$$

in a similar manner.

3.5.3. Alternative Checking by Reversing the Indexing

Another way to do the check is to take the image of the residual under the indexing relation, i.e. $\text{im}(H, R)[C, T]$. This will symbolically represent all the cases for which we know the property will hold.

$$\begin{aligned}
C, T \in \text{im}(H, R) &\Rightarrow \exists X(H[X, C] \wedge R[X, C, T]) \\
&\Rightarrow H[X^*, C] \wedge R[X^*, C, T] \text{ for some } X^* \\
&\Rightarrow s(C, T) \text{ by symbolic simulation invariant}
\end{aligned}$$

We can then check that $P \rightarrow \text{im}(H, R) \equiv \text{True}$. If so then the property is true. On some circuits and indexing relations, this method can avoid weak disagreements compared to the preimage method. This is particularly true if some cases in P are indexed by multiple X, C . However, if the number of indexing variables is much less than the number of target variables, this method can result in large BDDs that may be infeasible to compute.

3.5.4. Counter Example Analysis

Suppose that we have $P^R \wedge L \neq \text{False}$. Let (X, C) be such that $(P^R \wedge L)[X, C]$ is true. We show that (X, C) is a counter example to the property $\forall C \forall T (P[C, T] \rightarrow s(C, T))$.

	1.	$P^R[X, C] \wedge L[X, C]$	Premise
1	2.	$\text{dom}(R)[X, C] \wedge \forall T (R[X, C, T] \rightarrow P[C, T])$	Definition of P^R
2	3.	$\exists T (R[X, C, T])$	Definition of $\text{dom}(R)$
3	4.	Fresh T^* s.t. $R[X, C, T^*]$	
2, 4	5.	$P[C, T^*]$	
			Symbolic Simulation
	6.	$L[X, C] \wedge R[X, C, T^*] \rightarrow \neg s(C, T^*)$	Invariants
1, 3, 6	7.	$\neg s(C, T^*)$	
4, 7	8.	$P[C, T^*] \rightarrow \neg s(C, T^*)$	
8	9.	$\exists C, T (P[C, T] \rightarrow \neg s(C, T))$	
9	10.	$\neg \forall C \forall T (P[C, T] \rightarrow s(C, T))$	Property disproven

Figure 3: Proof for Positive Property Counter Example Check

It is practical to note that the set of counter examples we find is

$$\{(C, T) \mid \exists X (R[X, C, T] \wedge L[X, C] \wedge P^R[X, C])\}$$

which is described by the image operation on $P^R \wedge L$, $\text{im}(P^R \wedge L, R)$.

We can prove that the counter example analysis for negative properties is correct in a similar manner.

4. Partitioned Abstraction Relations

An important subclass of indexing relations that we consider is the partitioned indexing relation. A partitioned indexing relation is one that can be expressed in the following form:

$$R = \bigwedge (\text{hexpr} \rightarrow \text{expr} \wedge \text{lexpr} \rightarrow \overline{\text{expr}})$$

Where **expr** is either some **target variable** or an **expression of symbolic constants** while **hexpr** and **lexpr** are in terms of only the indexing variables.

In the implementation, a partitioned abstraction relation is represented as a list of tuples of the form:

$$[(\text{expr} = \text{targVar} / \text{Cexpr}, \text{hexpr}, \text{lexpr})]$$

The automatic abstraction algorithm produces partitioned abstraction indexing relations. The manually constructed indexing relations that we use in our examples are also partitioned abstractions.

This representation allows more efficient computation of preimages which are critical for allowing us to perform indexing transformations at scale.

4.1. Efficient Weak Preimage Computation

We first normalise R into

$$R = S[X, C] \wedge U[X, T]$$

where $U[X, T] = \bigwedge_{t_i \in \text{TargVars}} (h_i \rightarrow t_i \wedge l_i \rightarrow \overline{t_i})$

We can then make several observations.

Firstly,

$$\text{dom}(R)[X, C] = S \wedge \bigwedge_i \overline{h_i \wedge l_i}$$

Proof:

$$\begin{aligned}
\text{dom}(R)[X, C] &= \exists T R[X, C, T] \\
&= S[X, C] \wedge \exists T U[X, T] \\
&= S[X, C] \wedge \bigwedge_i (\exists t_i (h_i \rightarrow t_i \wedge l_i \rightarrow \bar{t}_i)) \\
&= S[X, C] \wedge \bigwedge_i (((h_i \rightarrow 1) \wedge (l_i \rightarrow 0)) \vee ((h_i \rightarrow 0 \wedge l_i \rightarrow 1))) \\
&= S[X, C] \wedge \bigwedge_I (\neg l_i \vee \neg h_i) \\
&= S \wedge \bigwedge_i \overline{h_i \wedge l_i}
\end{aligned}$$

Thus we are able to compute the domain of the indexing relation easily.

Secondly, to compute the preimage of some guard $P[C, T]$, we let $R \downarrow P$ denote that the part of the indexing relation that mentions target variables present in P . Specifically, if $\mathcal{F} = \text{FreeTargVars}(P)$ then

$$R \downarrow P = S[X, C] \wedge \bigwedge_{t_i \in \mathcal{F}} (h_i \rightarrow t_i \wedge l_i \rightarrow \bar{t}_i)$$

We have that $P_R = \text{dom}(R) \wedge P_{R \downarrow P}$

Proof:

$$\begin{aligned}
P_R[X, C] &= \exists T (R[X, C, T] \wedge P[C, T]) \\
&= \exists T \left(S[X, C] \wedge \bigwedge_{t_i \notin \mathcal{F}} (h_i \rightarrow t_i \wedge l_i \rightarrow \overline{t_i}) \wedge \bigwedge_{t_i \in \mathcal{F}} (h_i \rightarrow t_i \wedge l_i \rightarrow \overline{t_i}) \wedge P[C, T] \right) \\
&= S[X, C] \wedge \exists T_{\neg \mathcal{F}} \left(\bigwedge_{t_i \notin \mathcal{F}} (h_i \rightarrow t_i \wedge l_i \rightarrow \overline{t_i}) \right) \\
&\quad \wedge \exists T_{\mathcal{F}} \left(\bigwedge_{t_i \in \mathcal{F}} (h_i \rightarrow t_i \wedge l_i \rightarrow \overline{t_i}) \wedge P[C, T] \right) \\
&= S[X, C] \wedge \exists T_{\neg \mathcal{F}} \left(\bigwedge_{t_i \notin \mathcal{F}} (h_i \rightarrow t_i \wedge l_i \rightarrow \overline{t_i}) \right) \wedge \exists T_{\mathcal{F}} \left(\bigwedge_{t_i \in \mathcal{F}} (h_i \rightarrow t_i \wedge l_i \rightarrow \overline{t_i}) \right) \\
&\quad \wedge S[X, C] \wedge \exists T_{\mathcal{F}} \left(\bigwedge_{t_i \in \mathcal{F}} (h_i \rightarrow t_i \wedge l_i \rightarrow \overline{t_i}) \wedge P[C, T] \right) \\
&= S[X, C] \wedge \exists T \left(\bigwedge_{t_i \in \text{TargVars}} (h_i \rightarrow t_i \wedge l_i \rightarrow \overline{t_i}) \right) \\
&\quad \wedge \exists T_{\mathcal{F}} \left(S[X, C] \wedge \bigwedge_{t_i \in \mathcal{F}} (h_i \rightarrow t_i \wedge l_i \rightarrow \overline{t_i}) \wedge P[C, T] \right) \\
&= S[X, C] \wedge \bigwedge_i (\overline{h_i} \wedge \overline{l_i}) \wedge P_{R \downarrow P} \\
&= \text{dom}(R) \wedge P_{R \downarrow P}
\end{aligned}$$

We can further consider common cases of P that we will be constructing preimages for.

$$P_R = \begin{cases} \text{dom}(R) \wedge P & \text{if } \mathcal{F} \cap \text{TargVars} = \emptyset \\ \text{dom}(R) \wedge \overline{l_i} & \text{if } P = t_i \\ \text{dom}(R) \wedge \overline{h_i} & \text{if } P = \overline{t_i} \\ \text{dom}(R) \wedge P_{R \downarrow P} & \text{otherwise} \end{cases}$$

Proof:

If $\mathcal{F} \cap \text{TargVars} = \emptyset$, $R \downarrow P = \text{True}$ so

$$P_R = \text{dom}(R) \wedge \exists T (\text{True} \wedge P[C]) = \text{dom}(R) \wedge P[C]$$

If $P = t_i$, $R \downarrow P = (h_i \rightarrow t_i) \wedge (l_i \rightarrow \overline{t_i})$, so

$$P_R = \text{dom}(R) \wedge \exists t_i ((h_i \rightarrow t_i) \wedge (l_i \rightarrow \overline{t_i}) \wedge t_i) = \text{dom}(R) \wedge \overline{l_i}$$

The $P = \overline{t_i}$ case is analagous to the above.

4.2. Efficient Strong Preimage Computation

Observe that

$$\begin{aligned} P^R &= \text{dom}(R) \wedge \overline{\overline{P_R}} \\ &= \text{dom}(R) \wedge \neg(\text{dom}(R) \wedge P_{R \downarrow \overline{P}}) \\ &= \text{dom}(R) \wedge \neg P_{R \downarrow \overline{P}} \end{aligned}$$

This means that we don't need to do the final conjunction with $\text{dom}(R)$ when we compute a preimage if we are going to immediately be using the preimage to compute a strong preimage.

This represents an imporant speedup since we will be doing many strong preimage computations where computing $P_{R \downarrow \overline{P}}$ is easy (such as when $P = t_i$) but conjucting it with $\text{dom}(R)$ takes some time since $\text{dom}(R)$ can be complex.

5. Verification Under Environmental Constraints

Environmental constraints are constraints on the inputs to the circuit. They are of the form $J[C, T]$ to denote that we only need a constraint to hold if $J[C, T]$ is true. We call such a constraint a “care predicate”.

5.1. Index Over Care Predicate Cases via Indexing Relation

Restriction

Notice that one easy way to include environmental constraints is to simply add them to the guards of the output constraint. We can then check for the property holding under the enviromental constraint as described above.

However, this can lead to weak disagreements for abstractions³ that merge cases in J and cases in $\neg J$ such as the green assignment in Figure 5, since we cannot assume that J holds for the X, C indexing these cases.

This is referenced in [4] where it is recommended to find indexing relations that exactly index cases in J and not any in $\neg J$.

To get such an indexing relation, one idea is to have a new indexing relation that is the conjunction of the original indexing relation and the input constraint. This will ensure that the indexing relation only ever indexes target variable assignments satisfying the care

³Notably, if we don't use abstraction then this should always work

predicate, allowing the green assignment to be included in the strong preimage of a in Figure 5. However, this can still lead to weak disagreements from the indexing relation being too coarse.

An additional way to mitigate the weak disagreements is to use the alternative checking method involving reversing the indexing, described earlier. This can avoid weak disagreements in some cases but can be more computationally expensive.

5.1.1. Preserve Partitioned Indexing Relation by Conditioning Antecedent

Using the above procedure together with a the partitioned abstraction relation unfortunately destroys the partitioned structure, meaning we cannot directly apply the efficient preimage computations from Section 4.

However, we can employ an equivalent strategy where we first modify each BDD expression E in the antecedent to the form $E' := (E \wedge J) \vee \bar{J} \equiv J \rightarrow E$. We can then do the strong preimage computations, allowing us to only consider the indexing cases we know E to be true when the care predicate is true (corresponding to those cases that map exclusively to the bottom left 3 quadrants the T, C space of Figure 5). We modify the guard of the output constraint to include J and do the checking as described above.

Compared to restriction of the indexing relation, this approach will potentially cause some signals to be $\top$, but only on indexing variable assignments that exclusively index $\bar{J}$, such as the blue assignment in Figure 5. This is fine since J_R and J^R not will contain such cases, thus not affecting our output checking procedure for either correctness or counter example analysis. Since the only indexing cases that we consider during analysis are those that index at least one case in J , this is equivalent to to restriction method described above.

Since we are not modify the indexing relation, we can still use the efficient partitioned abstraction preimage operations.

5.2. Parametric Encoding

An alternative approach would be to use a parametric encoding [12] of the input constraints. A parametric encoding using the `param` function to compute a substitution of the input signals with functions of new parameterisation variables.

`param` takes a list of input constraints and a list of signals $s_1, s_2, \dots, s_n$ and computes a vector of boolean functions $f_1, f_2, \dots, f_n$ from new parameterisation variables $\mathbf{p} = \{p_1, \dots, p_k\}$ where $k \leq n$ for the purpose of substituting $s_i := f_i(\mathbf{p})$. These functions satisfy the following two conditions:

- (Soundness): $\forall \mathbf{p}, \langle s_i := f_i(\mathbf{p}) \mid i \in 1..n \rangle$ satisfies the input constraints
- (Completeness): $\forall \langle s_1, s_2, \dots, s_n \rangle$ that satisfy the input constraints, there exists some $\mathbf{p}$ such that $s_i = f_i(\mathbf{p})$

The intention of symbolic constants are such that they should not be abstracted or replaced, thus in our analysis of parametric encoding, we will assume that the symbolic constants are not part of the environmental constraint and thus not passed into `param`.

There are two ways to apply this to deal with environmental constraints.

5.2.1. Parametric Encoding of Indexing Relations

The first strategy is to apply the parametric encoding to transform an independently computed indexing relation. This was suggested in [4] but not proven sound. Given a circuit, antecedent, input and output constraints, we will

- Compute `param` on the input constraints
- Substitute each BDD variable in the antecedent with the corresponding function from the parametric encoding
- Compute an indexing relation, possibly via the automatic abstraction algorithm
- Substitute each BDD variable in the automatic abstraction result with the corresponding function from the parametric encoding
- Do the indexing transformation on the parameterised antecedent, and output constraints
- Do the symbolic simulation and check whether the output constraint holds

We prove this to be sound.

First we note that parameterising the antecedent is equivalent to bolting on the parameterisation functions onto each input to form a larger circuit. We can see that for each C, T such that $J[C, T]$ holds, we will have some C, T' that maps to it by the completeness of `param`.

We also note that the main base case for the symbolic simulation invariants still holds even though our input signals are now functions of the parameterisation variables. The input signals s are now replaced with the param functions f_s and our base case argument will still hold. This means that the symbolic simulation invariants will still hold.

Furthermore, we also have the parameterised indexing relation satisfying the coverage condition:

Assuming that the indexing relation $R[X, C, T]$ used satisfies the coverage condition $\forall T \forall C (J[C, T] \rightarrow \exists X R[X, C, T])$ then the parameterised indexing relation $R'[X, C, T']$ will also satisfy the coverage condition, in terms of the new parameterisation variables, i.e. $\forall T' \forall C \exists X R'[X, C, T']$ where T' is the new parameterised input signals.

	1. Fresh C, T'	
		Where f is the
<i>1</i>	2. Let $T = f(T')$	parametric encoding
<i>2</i>	3. $J[C, T]$	Soundness of param
	4. $\forall T \forall C (J[C, T] \rightarrow \exists X R[X, C, T])$	Premise
<i>3, 4</i>	5. $\exists X R[X, C, T]$	
<i>5</i>	6. Fresh X^* s.t. $R[X^*, C, T]$	
	7. $R' = R[T/f(T)]$	Definition of R'
<i>2, 6, 7</i>	8. $R'[X^*, C, T']$	Since C, T' were
<i>1, 8</i>	9. $\forall T' \forall C \exists X R'[X, C, T']$	arbitrary

Figure 4: Proof of Coverage Condition Satisfaction

The symbolic simulation invariants together with the coverage condition are sufficient premises for our proof of the output checking procedure to hold.

When considering the efficiency of this method, we note that destroys the partitioned structure of the indexing relation, meaning we cannot use the efficient preimage computations from Section 4.

We further show that this approach is equivalent to the indexing relation restriction method described in Section 5.1.

5.2.1.1. Equivalence to Indexing Relation Restriction

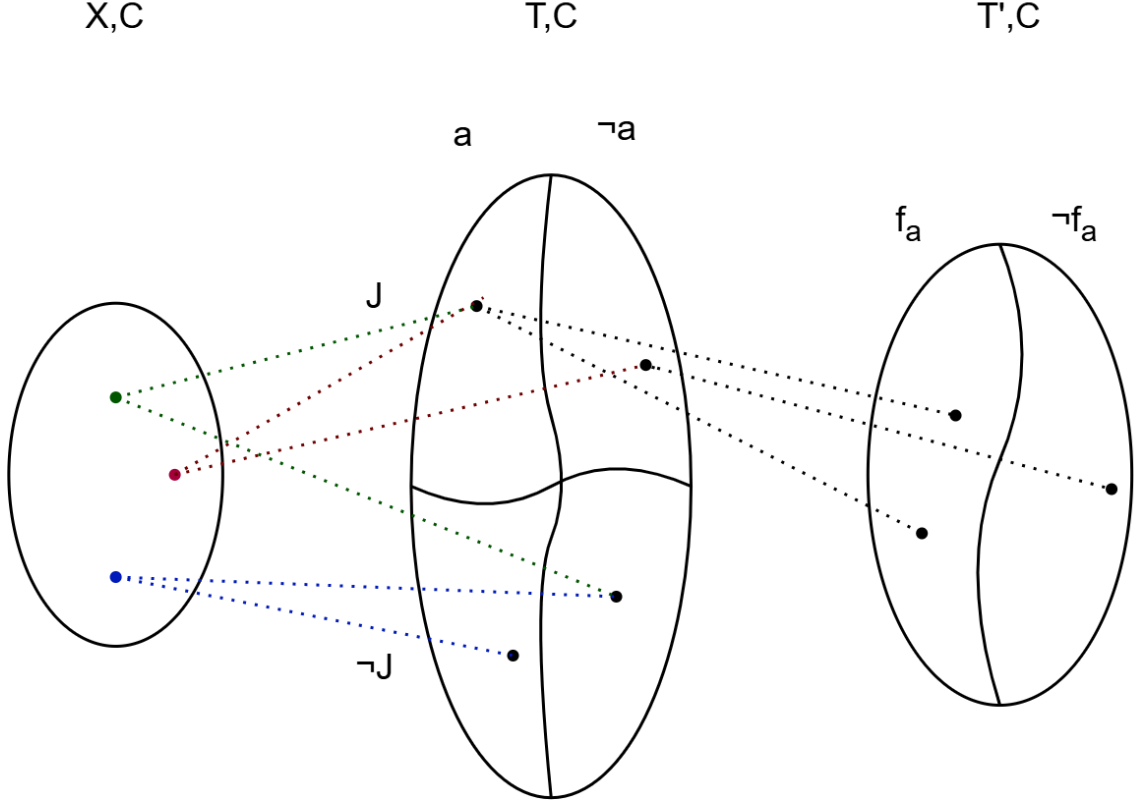


Figure 5: Illustration of Indexing Relation and Parameterisation

With careful observation, we note that the indexing cases that are included in the strong preimage operation on the antecedent are actually the same whether we are parameterising the indexing relation or restricting the indexing relation to the environmental constraint. Suppose we are taking the strong preimage of target variable a , we have that $a^R = (f_a)^{R'}$. In either case, we only consider the indexing cases that at least map to one target variable assignment that satisfies $J \wedge a$, and doesn't index any cases that satisfy $J \wedge \bar{a}$. Analysis is symmetric for taking the strong preimage of the negation of a target variable.

Furthermore,

$$\text{dom}(R') = \{X, C \mid \exists T' R'[X, C, T']\} = \{X, C \mid \exists T (P[C, T] \wedge R[X, C, T])\} = J_R$$

Both of them are going to the set of cases that index into at least one case in J . As such the output checks will be the same for both methods.

This means that both methods are actually equivalent, i.e. will produce the same results (proven, disproven or unproven).

Since the indexing relation restriction is equivalent to the more efficient antecedent conditioning, this parametric method is also equivalent to that. Given the better efficiency of the method of conditioning the antecedent, that is a preferable method.

5.2.2. Parameterise Before Abstraction

While the first strategy of using `param` is not any more effective than the restriction methods, our second strategy is likely to be more effective.

We will do the parameterisation first and then build an indexing relation on the circuit with the `param` functions bolted onto the front of the circuit, with the inputs of the larger circuit being the parameterisation variables. This will compute a new indexing relation that is not easily found as a modification of the non-parameterised indexing relation like we did in the restriction methods.

We then construct an indexing relation that satisfies the coverage condition over the parameterisation variables, possibly with the automatic abstraction algorithm. It is thus sound to use it for symbolic indexing. We modify the antecedent with the parameterisation functions substituted for the original signals and then transform them via the strong preimage under the indexing relation. We then run the symbolic simulation and then just check the output constraint without any guards. Soundness is similar to the proof in Section 5.2.1.

However, that would make it difficult to use the symbolic constants effectively since we would need to specify that in terms of the new input signals from the `param` substitution. If the input constraints don't involve the signals that were meant to be the symbolic constants, we can just `param` the rest of the inputs and then use the symbolic constants as is.

5.3. Conditioned Properties

A last idea is to move the environmental condition down to the SVA property level, adding conditions on the inputs to each SVA property.

However, this is equivalent to conditioning the output constraints with J , leading to the same possible issues of weak disagreements as described in Section 5.1.

Furthermore, if we are using the input constraints to case split on the possible inputs, we will need to programmatically modify the SVA properties since it would be impractical to do that manually.

6. Experiments and Evaluation

6.1. Content-Addressable Memory (CAM)

We illustrate the effectiveness of symbolic simulation with symbolic indexing on a stripped down version of a CAM.

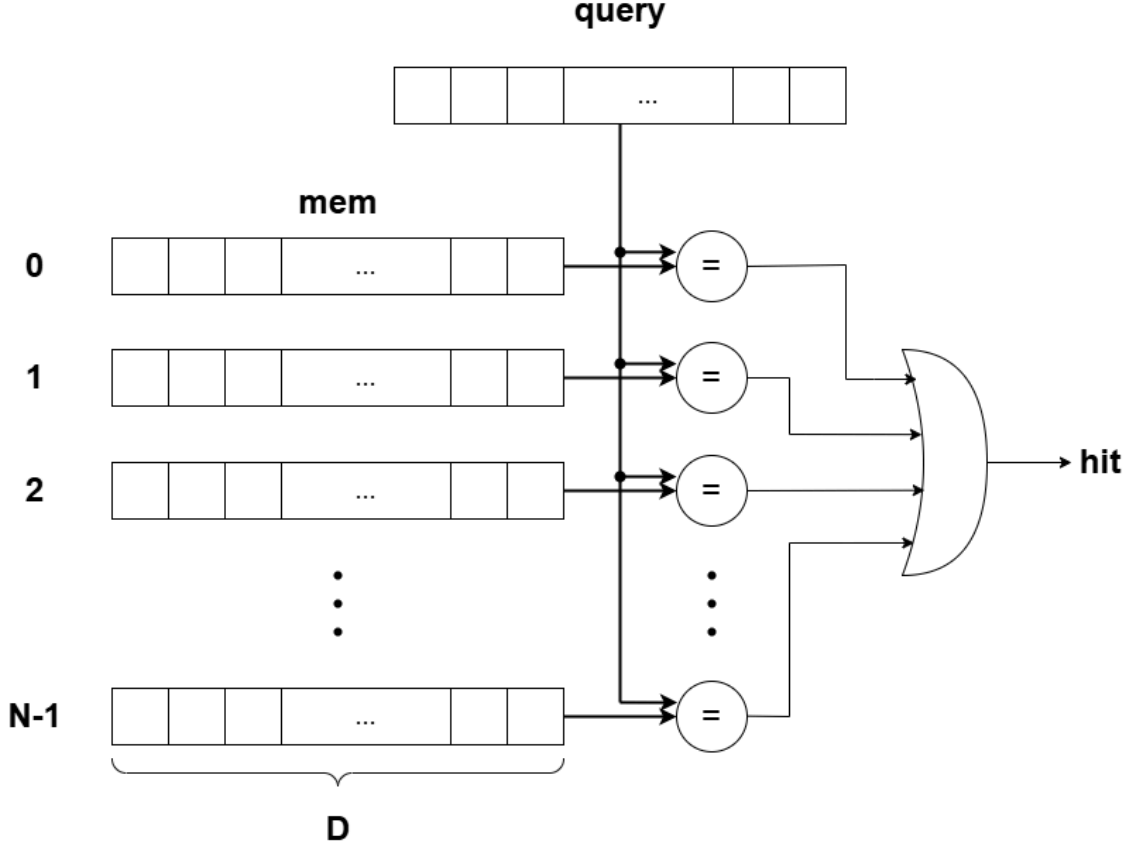


Figure 6: Content-Addressable Memory (CAM)

The CAM stores a fixed number of entries N , each being an integer of length D bits. The CAM also takes in a query of length D bits on each clock cycle and will output on a boolean wire whether the query matches with any of its entries. This wire is called the **hit** since it is high if there is a hit and low otherwise.

On each cycle our CAM will compare the query with each of the entries in parallel and take a logical OR of all the comparisons to get the **hit** value. The specification of our CAM does a similar operations, except it compares each entry with the query in sequence, doing many logical ORs in series to check for a hit.

Our property is that the specification and the circuit agree on the **hit** value.

6.1.1. Manual Indexing Relation

We construct a partitioned indexing relation manually that correctly provides symbolic indexing for the CAM to be verified via symbolic simulation with reference to [10].

The indexing relation encodes all the different ways that `hit` can be high or low. We will use the following indexing variables to effectively do case splitting:

- ***h***: Boolean variable
 - Whether or not the `hit` wire should be high in the case being considered
- ***ch***: Vector of $\log_2 N$ boolean variables
 - ***ch*** is interpreted as an integer which encodes which entry in the CAM should match with the query in the case being considered, only relevant if *h* is high.
- ***EM***: Matrix of size N by $\log_2 D$ boolean variables
 - Each ***em_i*** : $\text{em}_i[0], \text{em}_i[1], \dots, \text{em}_i[\log_2 D - 1]$ is interpreted as an integer to represent the bit of the *i*th entry of the CAM that should be different from the query, only relevant if *h* is low.

We also have the query ***q*** as a vector of symbolic constants, with length D .

We will add the following conjuncts together to get the indexing relation:

A conjunct that covers cases where the `hit` should be high:

$$\forall i \forall j : h \wedge (\mathbf{ch} = i) \Rightarrow (\mathbf{mem}[i][j] = \mathbf{q}[j])$$

A conjunct that covers cases where the `hit` should be low:

$$\forall i \forall j : \bar{h} \wedge (\mathbf{em}_i = j) \Rightarrow (\mathbf{mem}[i][j] \neq \mathbf{q}[j])$$

Where $0 \leq i < N$ and $0 \leq j < D$.

Equality between boolean vector encoded variables and the loop variables *i* or *j* are represented by a big conjunction of the individual bits or their negations to ensure the bit pattern formed from the whole vector matches that of the integer being compared to.

Equality between target variables *t* and indexing/symbolic constants *x* is done by adding 2 tuples of the form

- (*t*, (`premise AND x`), `false`)
- (`NOT t`, `false`, (`premise AND NOT x`))

to the indexing relation. This means that if the premise (such as $h \wedge (\mathbf{ch} = i)$) holds, then t will need to take the value of x . We can create analogous tuples for inequalities between target variables and indexing variables.

Observe that coverage will be achieved since for any given values of the query and the entries of the CAM **mem**, we will be able to find appropriate values for $h, \mathbf{ch}, \mathbf{em}$ that will satisfy the indexing relation.

Observe that this approach has $1 + \log_2 N + N \log_2 D + D$ indexing variables and symbolic constants. This is exponentially less than the ND variables that are considered without symbolic indexing. Considering this also lets us know that we should see maximum gain in efficiency of proving wider entry CAMs rather than CAMs with more entries.

6.1.2. Results

6.2. Multi-Input Maximum Circuit

We also showcase the method on a maximum circuit. The maximum circuit takes N inputs, each of length D , and every clock cycle, will output the maximum of all the inputs. This is implemented as a binary tree of 2-input maximum operations between tree nodes, that eventually leads to the output at the root of the tree.

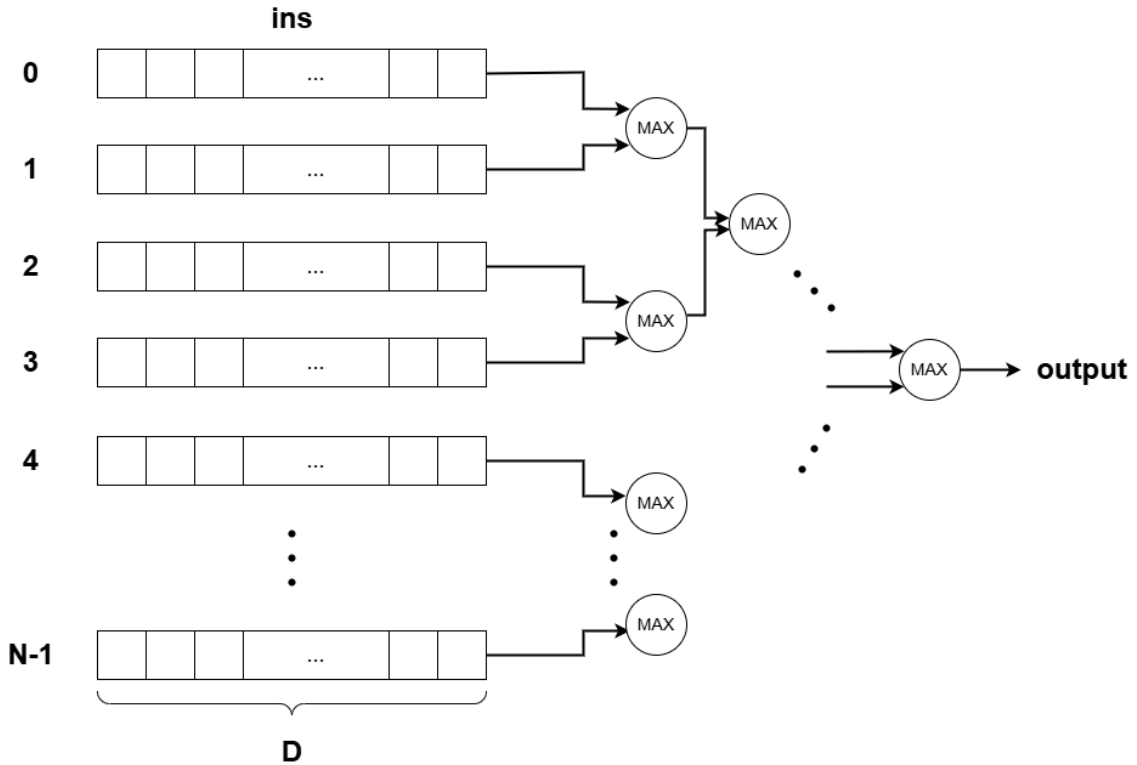


Figure 7: Multi-Input Maximum Circuit

Rather than using a specification that does the same computation in a different way, this circuit has a very natural relational specification that consists of two properties:

- Contained Property: The output of the circuit is one of the input values
- Bounded Property: The output of the circuit is at least as large as all the input values

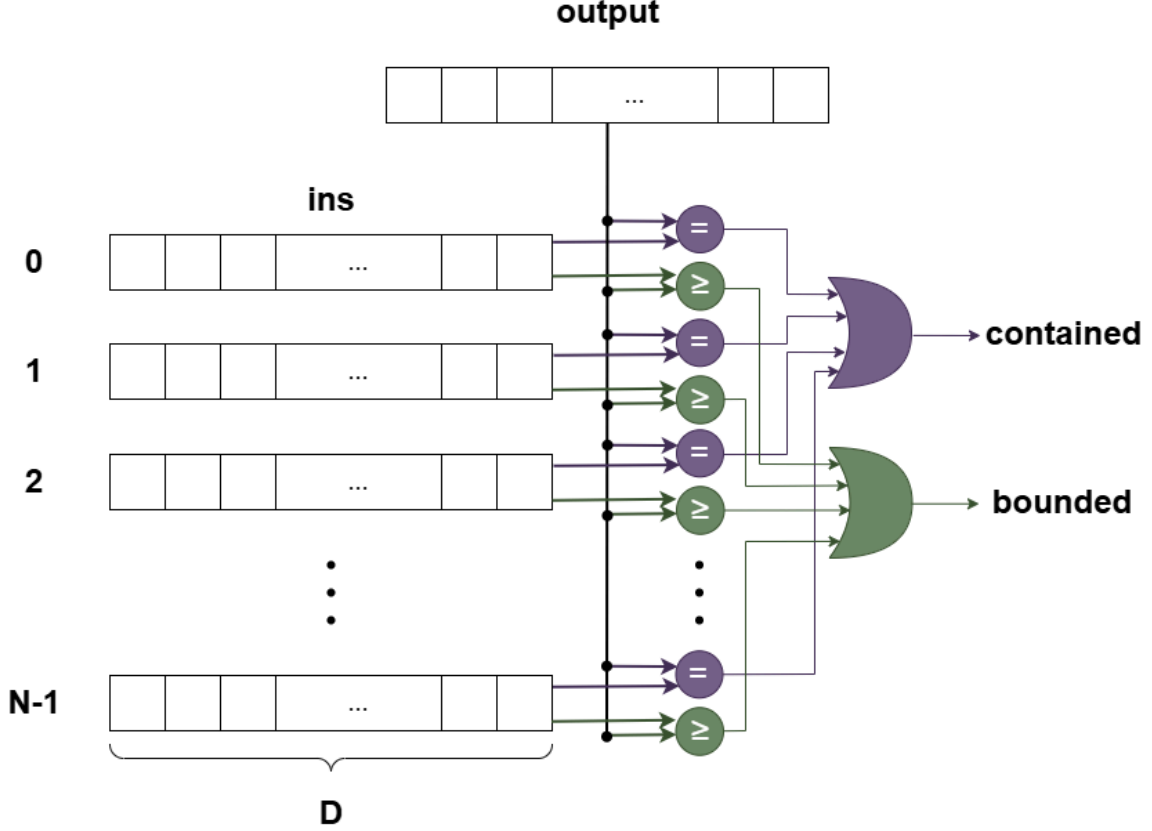


Figure 8: Maximum Circuit Specification Circuit

6.2.1. Manual Indexing Relation

Like for the CAM, we construct a partitioned indexing relation that will exponentially reduce the number of BDD variables needed for symbolic simulation.

We have the following indexing variables:

- $\mathbf{t}$: Vector of D boolean variables
 - $\mathbf{t}$ represents the target output of the maximum circuit. I.e. all cases that lead to the circuit producing x will be covered by some indexing cases where $\mathbf{t} = x$
- $\mathbf{d}$: Matrix of size N by $\log_2 D + 1$ boolean variables
 - $\mathbf{d}_i : \mathbf{d}_i[0], \dots, \mathbf{d}_i[\log_2 D]$ encodes an integer that represents the number of most significant bits of the i th input will be the same as the corresponding bits of $\mathbf{t}$ before the critical bit of i which will be low in $\mathbf{ins}[i]$ but high in $\mathbf{t}$.

We first write our required conjuncts in a manner that is easy to reason about, then convert them into a form that we can efficient encode as a partitioned indexing relation. Similarly, we will be having a few conjuncts that we will merge together:

A conjunct that ensures that at least one entry will completely match the target output:

$$\bigvee_{i=0}^{n-1} (\mathbf{d}_i = D)$$

A conjunct that ensures the most significant bits of each input match with the target output:

$$\forall i : \bigwedge_{j=D-\mathbf{d}_i}^{D-1} (\mathbf{ins}[i][j] = t[j])$$

A conjunct to ensure the critical bit for each entry is high in the target output but low in the input:

$$\forall i : \mathbf{d}_i < D \Rightarrow (\mathbf{ins}[i][D - \mathbf{d}_i - 1] = 0) \wedge (t[D - \mathbf{d}_i - 1] = 1)$$

Where $0 \leq i < N$.

We point out there is no restriction of the bits less significant than the critical bits in each input since their value will not affect the output, helping to achieve abstraction.

To actually form the partitioned indexing relation, we need to consider exactly the cases that we wish to force specific target variables to be high or low. This entail rewriting the second and third conjunct.

By rearranging $D - \mathbf{d}_i \leq j$ to get $D - j \leq \mathbf{d}_i$, we can rewrite the second conjunct as:

$$\forall i \forall j : (D - j \leq \mathbf{d}_i) \Rightarrow (\mathbf{ins}[i][j] = t[j])$$

Where $0 \leq j < D$.

By introducing j , we can also rewrite the third conjunct as:

$$\forall i \forall j : (D - 1 - j = \mathbf{d}_i) \Rightarrow (\mathbf{ins}[i][j] = 0) \wedge (t[j] = 1)$$

Constructing the actual partitioned indexing relation from the above rules follows a similar process to that done for the CAM. The only new formula is $\mathbf{y} \geq x$ for some boolean

encoded variable y and a loop variable x . To construct such a formula, we consider binary representation of x and use the following structural recurrence:

$$\begin{aligned} \text{geq}((y_0, \dots, y_{n-1}), (x_0, \dots, x_{n-1})) &:= (y_{n-1} > x_{n-1}) \vee \text{geq}((y_0, \dots, y_{n-2}), (x_0, \dots, x_{n-2})) \\ &\equiv (y_{n-1} \wedge \neg x_{n-1}) \vee \text{geq}((y_0, \dots, y_{n-2}), (x_0, \dots, x_{n-2})) \\ \text{geq}((y_0), (x_0)) &:= (y_0 = 1) \vee (x_0 = 0) \end{aligned}$$

This takes $D + N \log_2 D + N$ variables, exponentially less than the ND variables that are considered for symbolic simulation without symbolic indexing.

Further note that only the 2nd and 3rd conjuncts mention target variables. The first conjunct actually serves as a means of restricting the domain of our indexing relation ones where at least one d_i is equal to D . Compared to the manual indexing of the CAM, the domain in this case is not just True.

6.2.2. Results

7. Conclusion

We have reinvented the theory of rSTE and implemented symbolic indexing via indexing transformations for rSTE in JasperGold. We have shown that the method can be more effective than running symbolic simulations without any symbolic indexing.

7.1. Future Work

An area of major significance would be to develop practical algorithms for creating indexing relations specific to given environmental constraints. These indexing relations should index only into cases that satisfy the environmental constraints and be refined enough to avoid weak disagreements.

Automatic abstraction is critical to be able to apply rSTE widely. The speed of the automatic abstraction algorithm developed in the companion project is currently a limiting factor in efficient rSTE proofs. Furthermore, overabstraction is likely to be a problem in real world usage thus work on refinement of the automatically discovered indexing relations such as in [13] would make the rSTE much more usable.

Bibliography

- [1] S. Hazelhurst and C.-J. H. Seger, “Symbolic trajectory evaluation,” in *Formal Hardware Verification: Methods and Systems in Comparison*, T. Kropf, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 1997, pp. 3–78. doi: 10.1007/3-540-63475-4_1.
- [2] T. Melham, “Symbolic Trajectory Evaluation,” in *Handbook of Model Checking*, E. M. Clarke, T. A. Henzinger, H. Veith, and R. Bloem, Eds., Cham: Springer International Publishing, 2018, pp. 831–870. doi: 10.1007/978-3-319-10575-8_25.
- [3] R. E. Bryant, “Symbolic Boolean manipulation with ordered binary-decision diagrams,” *ACM Comput. Surv.*, vol. 24, no. 3, pp. 293–318, Sep. 1992, doi: 10.1145/136035.136043.
- [4] T. F. Melham and R. B. Jones, “Abstraction by Symbolic Indexing Transformations,” in *Proceedings of the 4th International Conference on Formal Methods in Computer-Aided Design*, in FMCAD '02. Berlin, Heidelberg: Springer-Verlag, 2002, pp. 1–18.
- [5] S. Adams, M. Bjork, T. Melham, and C.-J. Seger, “Automatic Abstraction in Symbolic Trajectory Evaluation,” in *Formal Methods in Computer Aided Design (FMCAD'07)*, 2007, pp. 127–135. doi: 10.1109/FAMCAD.2007.27.
- [6] C.-J. Seger *et al.*, “An industrially effective environment for formal hardware verification,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 24, no. 9, pp. 1381–1405, 2005, doi: 10.1109/TCAD.2005.850814.
- [7] C.-J. H. Seger, “VOSS - A Formal Hardware Verification System User’s Guide,” 1993. [Online]. Available: <https://api.semanticscholar.org/CorpusID:61246413>
- [8] J. O’Leary, R. Kaivola, and T. Melham, “Relational STE and theorem proving for formal verification of industrial circuit designs,” in *2013 Formal Methods in Computer-Aided Design*, 2013, pp. 97–104. doi: 10.1109/FMCAD.2013.6679397.
- [9] “Jasper RTL Apps.” [Online]. Available: https://www.cadence.com/en_US/home/tools/system-design-and-verification/formal-and-static-verification/jasper-verification-platform.html
- [10] M. Pandey, R. Raimi, R. Bryant, and M. Abadir, “Formal Verification Of Content Addressable Memories Using Symbolic Trajectory Evaluation,” in *Proceedings*

of the 34th Design Automation Conference, 1997, pp. 167–172. doi: 10.1109/DAC.1997.597138.

- [11] “Voss II.” [Online]. Available: <https://github.com/TeamVoss/VossII>
- [12] M. D. Aagaard, R. B. Jones, and C.-J. H. Serger, “Formal verification using parametric representations of Boolean constraints,” in *Proceedings of the 36th Annual ACM/IEEE Design Automation Conference*, in DAC '99. New Orleans, Louisiana, USA: Association for Computing Machinery, 1999, pp. 402–407. doi: 10.1145/309847.309968.
- [13] S. E. Adams, “Abstraction discovery and refinement for model checking by symbolic trajectory evaluation,” 2014. [Online]. Available: <https://www.cs.ox.ac.uk/people/tom.melham/phd/Adams-2013-ADR.pdf>